

TARGET — RAWAEA CENTRAL INVENTORY & STOCK MOVEMENT DESIGN

Target Design Reference — Inventory Domain

Status: DESIGN BASELINE — الكود إلى العودة قبل

Scope: Central Inventory + Manual Stock Vouchers + Direct Custody Movement

1. Purpose

Edge أي تعديل قبل اليدوية المخزنية والأذونات المخزون وحركات المركزي المخزون لنطاق المستهدف التصميمي المرجع هي الوثيقة هذه والاختبار التنفيذ ثم معه، والكود Schema ال توافق إثبات ثم أولاً، الصحيح النموذج تثبيت هو الهدف. Migration أو Function.

هو المعتمد المنهج: Understand → Prove → Design → Implement → Validate → Close.

هو المرجع. عليه للحفاظ مرجعًا خاطئ التاريخي السلوك يُعتبر لا لذلك عليها؛ الحفاظ يجب فعلية تشغيلية بيانات توجد ولا الإنشاء، تحت النظام Target Design، فعليًا موجود هو ما لإثبات Schema والكود استخدام مع.

2. Design Principles

- Inventory movement is a domain operation, not UI-owned logic.
- There is one central business core for stock movement. Applications initiate business intent; they do not implement independent inventory rules.
- A stock movement must produce its physical stock effect and inventory audit record atomically.
- Movement types are a closed domain vocabulary, not free-form text.
- allocated_qty is reservation state, not a physical stock movement, and is handled separately.
- Manual Stock Vouchers represent custody/stock movements between defined parties or locations; they are not a Runsheet.
- DirectSale is a custody-loading movement to a vehicle/representative, not the customer sale itself.
- The subsequent Van Sale consumes stock from the vehicle/representative custody.
- DirectReturn represents the return of remaining/returned custody stock from vehicle/representative to the warehouse.
- No application may become a second source of truth for stock quantities or movement history.

3. Central Inventory Business Core

الموثوق التنفيذ حد هو post_stock_movement نوع من مركزية بعملية ممثلًا المخزون، لحركات واحد Business Core هو الهدف الفيزيائية للحركات.

Responsibility	Target Owner
Validate movement request	Central Inventory Business Core
Check source availability	Central Inventory Business Core
Apply physical qty changes	Central Inventory Business Core
Write inventory movement/audit record	Central Inventory Business Core
Enforce movement semantics	Central Inventory Business Core + DB constraints
Reservation / allocated_qty	Separate reservation operation
UI workflow/state transitions	Voucher/domain functions subject to domain rules

Applications such as warehouse, van-sales, purchasing, returns, and voucher screens collect intent and request domain actions; they do not reproduce stock mutation logic.

4. Target Inventory State Model

Field	Target Meaning
stock_branches.qty	Physical quantity held by the branch/custody location.
stock_branches.allocated_qty	Reserved quantity; not itself a physical movement.
available_qty	Derived availability: physical quantity minus

	reserved quantity.
inventory_log	Auditable record of posted inventory movements.

No movement may treat allocated_qty as if it were a physical movement quantity.

5. Target Movement Vocabulary

Movement Type	Target Business Meaning
PurchaseIn	Stock enters inventory from purchasing/receiving.
TransferOut	Stock leaves the source location as part of a transfer.
TransferIn	Stock enters the destination location as part of a transfer.
Loading	Stock is loaded into operational custody/location when explicitly modeled as loading.
Unloading	Stock returns from operational custody/location when explicitly modeled as unloading.
POSSale	Point-of-sale sale consumes stock from the selling location.
VanSale	Vehicle/representative sale consumes stock from vehicle custody.
SalesReturn	Customer sales return adds stock back according to the return event.
PurchaseReturn	Stock returned to supplier leaves inventory.
InventoryIncrease	Authorized inventory increase.
InventoryDecrease	Authorized inventory decrease.
Adjustment	Authorized stock adjustment.

The exact use of Loading/Unloading versus TransferOut/TransferIn in a workflow must be fixed by the domain event contract before implementation; applications may not invent alternate movement semantics.

6. Manual Stock Voucher Domain

Manual Stock Voucher is a formal document for moving stock/custody between defined parties or locations. It is not part of the Runsheet domain.

Voucher Type	Target Meaning
Transfer	Movement from one branch/location to another.
DirectSale	Direct custody issue: warehouse → vehicle/representative custody for field selling; not the customer sale.
DirectReturn	Direct custody return: vehicle/representative custody → warehouse.
SupplierReturn	Stock returned from warehouse to supplier.
Scrap	Authorized disposal/destruction movement.
Adjustment	Authorized stock adjustment/count reconciliation.

7. Voucher Lifecycle

Create → Send → Receive → Complete → Cancel

Stage	Target Responsibility	Inventory Effect
Create	Create a valid voucher in a pre-posting state.	No physical stock movement.
Send	Validate and initiate the movement workflow.	Only the defined posting event may create the movement.
Receive	Receiving party confirms receipt where required.	Only the defined receiving event creates the receiving-side effect.
Complete	Finalize the document after required confirmations.	No second physical movement merely because status becomes Complete.
Cancel	Cancel only when business rules permit.	No movement if cancelled before posting; reversal semantics must be explicitly defined after posting.

A status transition must not accidentally become a second stock movement. Each physical movement is posted once by the central inventory engine.

8. Target Flows

8.1 Transfer

Source Branch → Transfer Voucher → Destination Branch. Source decreases and destination increases, with an auditable movement record. The two sides belong to one business transaction/effect.

8.2 Direct Sale / Custody Issue

Warehouse → DirectSale Voucher → Vehicle/Representative Custody → Field Sale

DirectSale is the custody-loading event, not the customer invoice. The vehicle/representative is temporary operational stock custody. The later Van Sale is a separate sales event consuming stock from that custody.

8.3 Direct Return

Vehicle/Representative Custody → DirectReturn Voucher → Warehouse

DirectReturn is the return of remaining/returned custody stock and is distinct from a customer sales return.

8.4 Supplier Return

Warehouse → SupplierReturn → Supplier

Supplier return is an inventory-out event with its own business/accounting consequences; UI code does not implement independent stock mutation.

9. Source of Truth

Domain	Target Source of Truth
Physical branch/custody quantity	stock_branches.qty
Reserved quantity	stock_branches.allocated_qty
Available quantity	Derived from physical minus reserved quantity
Posted inventory movement history	inventory_log
Voucher document/header	stock_vouchers

Voucher line quantities	stock_voucher_details
Customer/sales transaction	Sales domain tables (outside this movement-engine scope)
Runsheets	Runsheets domain (outside Manual Voucher ownership)

Detailed accounting balance ownership is outside this inventory design unless explicitly included by a later approved accounting Target Design.

10. Atomicity and Transaction Boundary

A posted inventory movement must be atomic: validation, stock mutation, and inventory movement recording succeed together or fail together.

The target implementation boundary is a database-level atomic operation/RPC represented by `post_stock_movement`. The application layer must not split the physical update and inventory log into unrelated requests.

The central operation must use appropriate row locking/concurrency control so simultaneous movements cannot both consume the same physical quantity.

11. Reservation Boundary

`allocated_qty` is not a substitute for posting a movement. Reservation and release are separate domain actions. The movement engine changes physical stock; a separate reservation operation manages `allocated_qty`.

12. Application Boundaries

Application / Domain	May Do	Must Not Own
Manual Stock Voucher UI	Collect voucher data; request lifecycle actions.	Independent stock mutation rules.
Van Sales	Perform van-sales workflows and request domain operations.	Independent inventory movement rules.
Warehouse	Perform warehouse workflows and invoke approved inventory actions.	A second inventory engine.
Purchasing	Initiate purchase receipt/return business events.	Ad-hoc stock mutations.
Returns	Initiate approved return events.	Independent stock truth.

13. Accounting Boundary

Inventory movement and accounting are related but distinct domain responsibilities. The inventory engine owns the physical stock effect and inventory audit. Accounting posting must occur through the approved accounting domain engine and must not be duplicated across voucher, sales, settlement, and UI functions.

This document deliberately does not freeze detailed chart-of-accounts mappings or journal-line rules; those belong to the accounting Target Design and must be proven before implementation.

14. Van Sales Boundary

- DirectSale = load custody.
- VanSale = sell from custody.
- DirectReturn = return remaining/returned custody.
- Settlement = reconcile the operational result of van-sales activity.

These are separate business events. Van Sales must not interpret DirectSale as a customer sale. Runsheet is a separate operational domain and must not become the hidden owner of DirectSale/DirectReturn custody movements.

15. Core Invariants

- Every physical stock mutation has one authoritative inventory posting path.
- Every posted movement has an auditable inventory_log record.
- A movement is never posted twice because two voucher statuses changed.
- DirectSale never means customer sale; it means custody issue.
- DirectReturn never means customer sales return; it means custody return.
- VanSale consumes vehicle/representative custody; it does not create that custody.
- allocated_qty is reservation state, not physical movement.
- Applications cannot become alternative sources of truth for qty.
- Runsheet does not own Manual Stock Voucher semantics.
- No implementation proceeds until required Schema and code evidence is mapped to this target.

16. Validation Contract Before Implementation

- Map all inventory-affecting Edge Functions to the target movement vocabulary.
- Identify all writes to stock_branches.qty.
- Identify all writes to stock_branches.allocated_qty separately.
- Identify all writes to inventory_log.
- Identify all voucher lifecycle functions.
- Identify DirectSale and DirectReturn end-to-end.
- Identify VanSale stock consumption separately from custody loading.
- Separate Runsheet movements from Manual Voucher ownership.
- Map accounting side effects without assuming current implementation is the target.
- Prove concurrency and atomicity behavior before implementation.

Anything not proven from Database Schema, Edge Functions, applications, or explicit approved Target Design remains UNKNOWN and is not filled by assumption.

17. Implementation Gate

No Edge Function is modified merely because it currently behaves incorrectly. Modification occurs only after its intended event, source of truth, movement type, transaction boundary, inputs, outputs, and side effects are proven against this target design.

Target Design → Evidence Matrix → Schema Verification → Function Mapping → Implementation → Test Contract → Validation → Close

18. Explicit Out-of-Scope

- Detailed sales-order architecture.
- Detailed Runsheet architecture.
- Full accounting chart and journal design.
- Driver ledger target design.
- UI redesign.
- PWA implementation details.
- Historical data migration/cleanup.
- Performance tuning unrelated to the inventory movement boundary.

19. Design Decision Summary

Decision	Target
Inventory core	One central business core.
Physical movement authority	Central inventory movement operation.
Physical stock truth	stock_branches.qty
Reservation truth	stock_branches.allocated_qty
Movement audit	inventory_log
Voucher header	stock_vouchers
Voucher details	stock_voucher_details
Voucher lifecycle	Create → Send → Receive → Complete → Cancel
DirectSale	Custody issue to vehicle/representative.
VanSale	Separate sale consuming custody stock.
DirectReturn	Custody return to warehouse.
Runsheet	Separate operational domain.
Implementation method	Prove → Design → Implement → Validate.

20. Final Target Principle

RAWAEA inventory must have one authoritative business core for physical stock movement. Applications express business intent; they do not own independent inventory logic. Manual Stock Vouchers manage formal custody/stock movements, while Van Sales consumes and reconciles the custody created by those movements. The target is established before code repair so no function is repaired against an unstable or accidental business model.

21. Evidence Basis

- Architecture/اليدوية المخزنية الأذونات.md
- Edge_Function_Reports/_HISTORICAL/van-sales report.md
- Previously established project Target Design direction for a central inventory movement engine.
- Explicit project instruction that the system is under construction and Target Design is the decision reference.

Where the references do not establish a final implementation detail, this document deliberately leaves the detail open for the subsequent Prove stage rather than inventing a rule.